FARMPULSE AI · ARCHITECTURE PROPOSAL A

FarmPulse Edge

An on-farm, edge-first modular monolith for cow monitoring, suspicion flagging and prioritised alerts

VERSION	AUDIENCE	STATUS
1.0 · 24 October 2026	FarmPulse AI team, supervisors, farm partner	Proposal for review

The idea in one line

Run the whole pipeline (video analysis, per-cow records, suspicion classifier, prioritisation, alerts and dashboard) on one GPU server installed at the farm, built as a single Python codebase with clear internal modules.

Who it is for

The current stage in the project README: vision only, one farm, a four-student team, milestones M1 to M4, limited budget and an unreliable rural internet link.

Why this shape

Video never leaves the farm, alerts still work offline (GSM fallback), the existing FastAPI and YOLO11 code carries straight over, and running cost stays at roughly $20–70 a month after a one-off hardware spend.

What it gives up

Multi-farm scale, central model training on live data, and elastic compute. The design keeps the event schema and identifiers compatible with Proposal B so the system can grow into it later.

1 Context and scope
2 Architecture drivers
3 Architecture overview
4 Components
5 Data flow and data model
6 AI and ML design
7 Technology stack
8 Trade-offs and decisions
9 Risks and mitigations
10 Scalability and performance
11 Security and privacy
12 Cost estimate
13 Rollout plan
14 Evaluation and success criteria
A Mapping to the existing code

1 Context and scope

FarmPulse AI aims to understand each animal continuously and turn changes in its behaviour into early, reviewable warnings. The project proposal (*FarmPulse_AI_Final.pdf*) describes the full vision: cameras, microphones and farm records feeding an animal digital profile, a multimodal temporal fusion model, early-risk alerts, herd analytics and a digital-passport marketplace. The project README narrows the **current stage to vision only**: capture video, detect behaviour and visible state, store a time-stamped record per cow, classify each cow as suspicious or not with an estimated time window, rank suspicious cows with an RL prioritisation module, and alert the owner or vet through the dashboard, SMS and WhatsApp. Audio is listed as future work.

Proposal A is designed for that current stage. It answers the proposal's core research question with the smallest system that can be deployed and evaluated on a real farm:

> Can continuous visual behaviour analysis, combined with each animal's historical baseline, provide useful early warnings of abnormal conditions and help farmers find the animals that need inspection?

Where the project stands today

Asset in the folder	What it already does	Gap this architecture closes
`cow-detection/`	YOLO11 cow detection (COCO class 19), BoT-SORT/ByteTrack tracking tuned for cattle, offline ID repair (`relink.py`), zone-based behaviours (eating, drinking, lying, walking, standing), FastAPI dashboard, CSV/JSON exports. Runs on recorded video files.	Live camera streams, persistence across days, per-cow identity, alerts.
`cow_hide_identification/`	Siamese ResNet-50 network with 128-d embeddings, triplet loss and semi-hard mining, ready to train once labelled crops exist. No dataset or trained model yet.	A herd gallery and an identity resolver that uses the embeddings in production.
Videos and `training_frames.zip`	Handheld WhatsApp clips and fixed-camera CCTV footage (2304×1296, Hikvision-style files), with zones drawn for the 15 Oct camera.	A labelled, versioned dataset from fixed cameras, including nights and crowded feeding.
Reference papers (13)	Holstein re-ID (OpenCows2020, MultiCamCows2024, dense crowds), behaviour datasets (CBVD-5, Cattle-CLIP), YOLO behaviour detectors, DeepSORT/StrongSORT cattle tracking, cattle vocalisation, IoT health models, PPO for herd management.	Turn the research into a staged, testable system.

In scope for Proposal A

- One farm, 4 to 8 fixed IP cameras covering feed trough, water trough, resting area and the pen entrance.
- Continuous video analysis: detection, tracking, identity, behaviour, visible state (for example mouth open) and an intruder detector.
- Per-cow records at a fixed interval, personal behaviour baselines, a suspicion flag with an estimated time window, a priority and suggested action, and alerts over dashboard, WhatsApp and SMS.
- Weekly reports, feedback capture from owner and vet, and an offline training loop on a separate workstation.

Out of scope (deferred, with a defined extension point)

- Audio and vocalisation analysis (a slot exists in the perception worker and the record schema).
- Multiple farms, central cloud analytics, marketplace and digital passport (these move to Proposal B or a later phase).
- Any automated diagnosis or treatment. Every output is decision support that a person reviews.

2 Architecture drivers

Driver	Requirement	How Proposal A responds
Connectivity	Rural farms often have weak or intermittent uplinks. A 4 MP stream needs about 4 Mbps, so 8 cameras would need over 30 Mbps of constant upload.	All inference on site. Only alerts, summaries and backups leave the farm.
Alert latency	Intruder alerts in seconds; health alerts within minutes of the evidence.	A separate security fast path (under 30 s). Health pipeline closes a 5-minute interval and alerts within 15 minutes.
Team capacity	Four students, one academic year, Python and PyTorch skills already in use.	One codebase, one database, Docker Compose. No Kubernetes, no message broker.
Budget	Graduation-project budget, no committed cloud spend.	One-off hardware of about $2,100–3,600; running cost of about $20–70 a month.
Data honesty	The README requires keeping measured, predicted and confirmed data separate and avoiding claims before evaluation.	Separate tables for observations, predictions, alerts and outcomes; every model output is versioned.
Privacy	Footage shows farm workers and the owner's premises.	Video stays local; person regions are blurred in stored clips; short retention by default.
Evolvability	The proposal's audio, fusion and passport features will come later.	Stable record schema with optional modality fields and IDs compatible with Proposal B.

3 Architecture overview

Proposal A is a **modular monolith on an edge server**. One Python package (`farmpulse`) contains every module with explicit interfaces between them, and it runs as a handful of processes under Docker Compose: an ingest process, a GPU perception worker, a security worker, an application process (records, classifier, prioritiser, dispatcher, API) and PostgreSQL. Modules talk through PostgreSQL tables used as durable work queues (`SELECT … FOR UPDATE SKIP LOCKED`) rather than through a broker. That keeps one source of truth and one thing to back up.

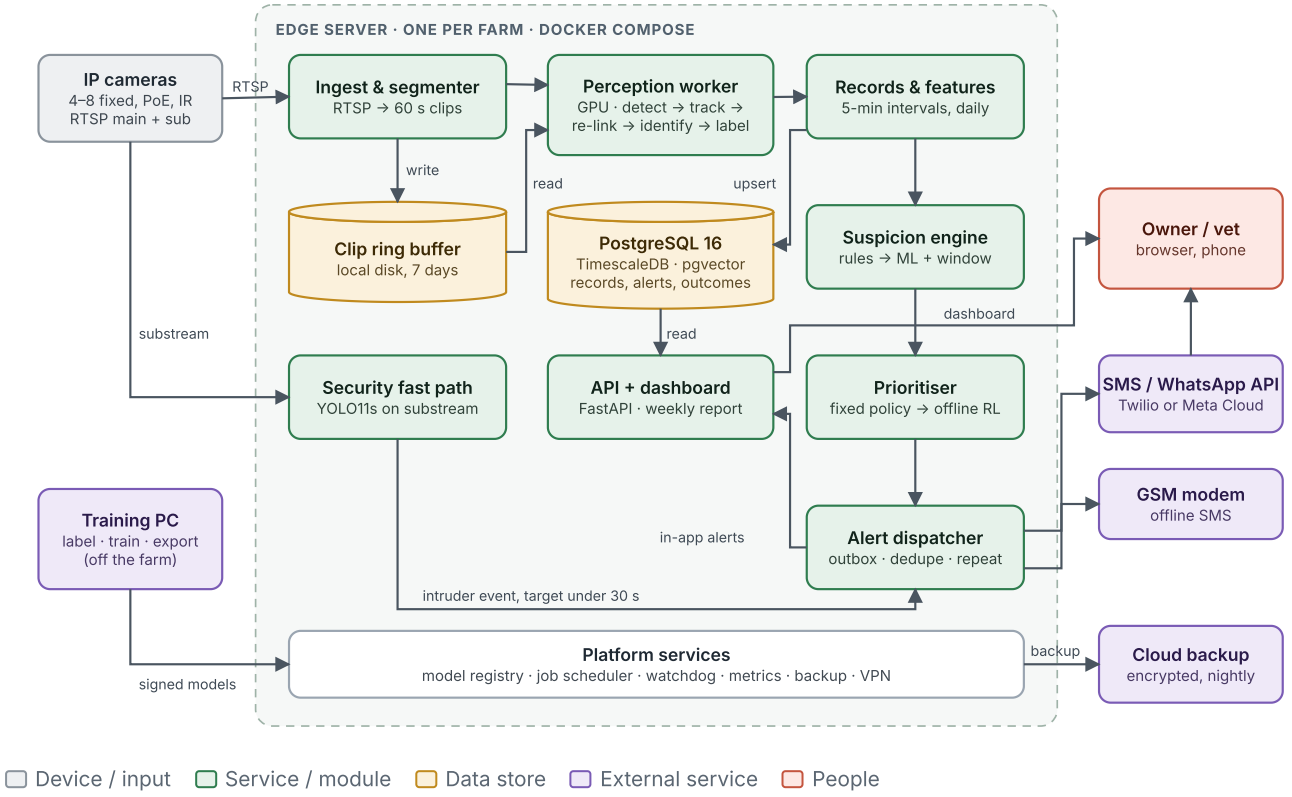


Figure 1. Proposal A component view. Everything inside the dashed box runs on one machine at the farm. The only outbound traffic is alerts, encrypted backups and the admin VPN.

Key architectural choices

- **Two paths with different latency.** The health path is batch-oriented: it processes completed 60-second clips, because behaviours such as eating (3 s dwell) and lying (rolling median) need context, and minutes of latency are acceptable. The security path runs a small detector on each camera's low-resolution substream so an intruder alert does not wait for the batch.
- **Identity is tiered, not assumed.** A tracker number is not an identity. The identity resolver combines ear-tag reading at chokepoints, hide-pattern embeddings from the existing Siamese model and track continuity. When confidence is low, observations are stored as *unassigned*, as the README requires.
- **Rules first, learning later.** The suspicion engine and the prioritiser both start as transparent, testable rules. They are replaced by learned models only when verified outcomes exist, and the rule version is kept as the baseline in every comparison.
- **The database is the integration point.** TimescaleDB stores time series, pgvector stores the herd embedding gallery, and ordinary tables hold alerts, feedback and the transactional outbox for notifications.

4 Components

Component	Responsibility	Inputs → outputs	Built from
Ingest & segmenter	Pull RTSP from each camera, cut 60 s H.264/H.265 segments without re-encoding, stamp camera, zone and NTP time, and watch stream health.	RTSP → clip files + `clips` rows (status *ready*)	ffmpeg segment muxer, PyAV; replaces `frame_extraction.py`
Clip ring buffer	Keep full footage for 7 days; keep event clips (alerts, low-confidence, labelling samples) for 90 days.	Clips → disk, with retention job	Local HDD; optional reuse of the existing NVR
Perception worker (GPU)	Per clip: detect cows at 3–5 fps, track, re-link fragments, resolve identity, label behaviour and visible state. Writes per-clip detections to Parquet and tracklet summaries to PostgreSQL.	Clip → tracklets (cow_id or unassigned, behaviour seconds, state events, confidence)	`detector.py`, `tracker.py`, `relink.py`, `behaviour.py`, `zones.py`, `cowid`
Identity resolver (inside perception)	Match tracklets to registered cows: tag OCR when visible, otherwise nearest neighbour in the herd gallery with a calibrated threshold, otherwise unassigned. Queue doubtful matches for a person to confirm in the dashboard.	Tracklet crops → cow_id + method + score	`cowid.inference.CowEmbedder`, pgvector, new tag detector
Security fast path	Detect people, dogs and vehicles in the pen zones outside working hours (and other configured rules). Send a snapshot alert straight to the dispatcher.	Substream frames → `security_events`	YOLO11s COCO classes; zone polygons
Records & feature builder	Roll tracklets into one record per cow per 5-minute interval, then daily features: feeding minutes, drinking visits, lying minutes and bouts, walking minutes, time apart from the herd, and data-coverage indicators.	Tracklets → `cow_observations`, `cow_daily_features`	New; SQL continuous aggregates
Suspicion engine	Compare each cow with its own rolling baseline; output suspicious or not, an estimated time window and confidence. Rule set v0, then a supervised model v1.	Features + history → `predictions`	New; scikit-learn / LightGBM
Prioritiser	Turn suspicious cases into priority (low, medium, high), urgency and a suggested action from an approved list. Fixed policy v0, offline-trained policy v1 in shadow mode.	Prediction + context → `decisions`	New; d3rlpy for v1
Alert dispatcher	Channel rules by priority, de-duplication, quiet hours, repeat until acknowledged, delivery receipts. Uses a transactional outbox so an alert is never lost or sent twice.	Decisions / security events → `alerts`, SMS, WhatsApp	New; Twilio or WhatsApp Cloud API, Gammu for GSM

API + dashboard	Herd overview, cow profile timeline, alert inbox with confirm and reject, identity review queue, zone editor, clip playback, weekly report.	DB → browser (LAN or VPN)	Existing `app.py` and `static/`
Platform services	Model registry (versioned, checksummed files), job scheduler, process watchdog, metrics, nightly encrypted backup, admin VPN.	-	APScheduler, Prometheus, Grafana, restic, WireGuard

5 Data flow and data model

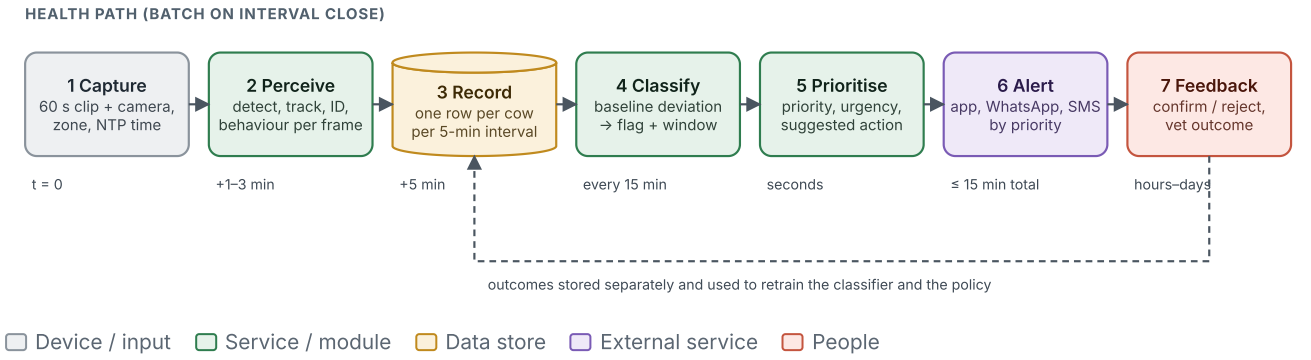


Figure 2. Health path from capture to feedback, with target latencies. The security path skips steps 2 to 5 and alerts within 30 seconds.

1. **Capture.** The segmenter closes a 60 s clip and inserts a `clips` row. Camera clocks are synced by NTP from the edge server, which fixes the clock-alignment task in the README.
2. **Perceive.** The perception worker claims the clip, samples 3–5 fps, runs detection and BoT-SORT with the existing cattle settings, re-links fragments, then resolves identity per tracklet. Behaviour labels come from the zone rules and, once trained, the posture classifier.
3. **Record.** When a 5-minute interval closes, the builder writes one row per identified cow (seconds per behaviour, state events, zones visited, coverage) and keeps unassigned tracklets in their own table.
4. **Classify.** Every 15 minutes, and at the end of each day, the suspicion engine scores cows whose data changed and writes a `predictions` row with model version, features used and the estimated window.
5. **Prioritise.** Suspicious cows get a decision: priority, urgency and one approved action (observe again, request a check today, call the vet now).
6. **Alert.** The dispatcher sends by channel rules: high goes to dashboard, WhatsApp and SMS and repeats every 30 minutes until acknowledged; medium goes to dashboard and WhatsApp; low goes into the daily digest.
7. **Feedback.** The owner or vet confirms or rejects and later records the outcome. Outcomes are stored apart from predictions and become training labels.

Logical data model

Table	Contents	Type and retention
`cows`	cow_id, tag number, name, breed, sex, birth date, coat type (patterned or solid), status	Relational, permanent
`cow_gallery`	cow_id, view (left, right, top), 128-d embedding, source crop, model version	pgvector, permanent

`cameras`, `zones`	RTSP URL (secret reference), calibration such as `standing_height`, zone polygons	Relational; mirrors `zones/*.json`
`clips`	camera, start and end time, path, status, processing version	Relational; rows kept, files follow retention
`tracklets`	clip, track id, cow_id or null, id method and score, behaviour seconds, states, bbox stats	Hypertable, 1 year
`cow_observations`	cow_id, interval start, zone, behaviour seconds, state flags, coverage, source tracklets	Hypertable, compressed after 30 days
`cow_daily_features`	daily aggregates and baseline deviations (robust z-scores)	Continuous aggregate
`predictions`	cow_id, time, suspicious, target outcome, window hours, confidence, model version, feature snapshot	Relational, permanent
`decisions`, `alerts`	priority, urgency, action, policy version, channels, delivery status, acknowledgement	Relational, permanent
`feedback_outcomes`	who, when, confirm or reject, vet finding, confirmed condition and date	Relational, permanent
`security_events`, `weekly_reports`	snapshot path, class, zone; weekly aggregates	Relational

Per-frame detections (thousands of rows per minute) are not stored in PostgreSQL. They go to one Parquet file per clip, like today's `detections.csv`, so the database stays small: 100 cows × 288 intervals a day is under 30,000 observation rows a day.


```
{
  "cow_id": "C104", "interval_start": "2026-11-02T06:05:00Z", "zone": "feed",
  "behaviour_s": {"eating": 212, "standing": 61, "lying": 0, "walking": 12, "drinking": 0},
  "states": ["mouth_open"], "coverage": 0.95, "id_method": "gallery", "id_score": 0.86,
  "source": {"camera": "cam-03", "clips": ["cam-03/2026-11-02T06-05.mp4"], "pipeline": "perc-1.4.0"}
}
```

6 AI and ML design

The four student work packages in the proposal map onto modules of the monolith. Because audio is outside the current stage, the third package focuses on visible state and security, and moves to audio when microphones are added.

Package	Module	Approach	Data
AI 1	Detection, tracking, identity	Fine-tune YOLO11m on farm frames (crowded feeding line, lying cows at night), keep BoT-SORT + re-link. Identity tiers: ear-tag detector + OCR at feed and water cameras; Siamese hide embeddings (`cowid`) against the herd gallery; human confirmation for doubtful cases. Dense-crowd segmentation (detect-segment-identify, 2602.15962) as a stretch goal.	Farm footage, OpenCows2020, MultiCamCows2024
AI 2	Behaviour	Keep the zone and geometry rules as v0. Add the posture classifier (`train_posture.py`) once night footage gives enough lying examples. Then a clip-level temporal model on tracklet crops (for example X3D-S or a Cattle-CLIP style video-language model) for rumination and grazing.	Farm clips, CBVD-5, CattleBehaviours6
AI 3	Visible state and security	Per-tracklet state classifier (mouth open, abnormal posture, lameness cues as a stretch). Security uses COCO classes with zone and time rules. Audio is added later through the same tracklet and record interface.	Farm crops; audio datasets later
AI 4	Suspicion + prioritisation	See below.	Records + vet outcomes

Suspicion engine

- **v0 rules (from week one of data).** For each cow, compute a 7-day rolling median and median absolute deviation for each daily feature, excluding the last 24 hours. Flag when, for example, feeding time falls below baseline by more than 3 robust deviations and lying time rises, sustained over two 6-hour windows, with enough coverage. Thresholds are set with the vet. The window is a fixed statement ("check within 24–72 h").
- **v1 supervised model (once outcomes exist).** Gradient-boosted trees on windowed features plus deviations, with a discrete-time survival head (hazard for 24, 48 and 72 h) to give the time window. Splits are by cow and by period to avoid leakage. Report recall, false alerts per 100 cow-days and lead time against v0.

Prioritisation (the RL module)

- **v0 fixed policy.** Priority = f(suspicion confidence, window, coverage, cow context such as recent calving), mapped to approved actions. This is the baseline the README asks the RL policy to beat.
- **v1 offline policy.** Log every decision, the action taken and the outcome. Train a contextual bandit first, then a conservative offline RL method (IQL or CQL) only if there are enough logged episodes. Evaluate with off-policy estimators and run in shadow mode next to v0. A person approves every real action, and there is no online self-training from unverified alerts.

Training and deployment loop

Training happens off the farm on a GPU workstation or university or Colab GPUs. Labelling uses CVAT or Label Studio on clips exported from the edge (event clips plus a random sample). Datasets are versioned with DVC and runs are tracked in MLflow. A model is exported to TensorRT (FP16) or ONNX, checksummed and signed,

then copied to the edge and registered. The perception worker loads models by registry version, so every record can be traced to the model that produced it.

7 Technology stack

Layer	Choice	Reason
Edge hardware	Small-form desktop with RTX 4060 Ti 16 GB or RTX 4070, 32 GB RAM, 1 TB NVMe + 4 TB HDD. Alternative: Jetson AGX Orin for a sealed, low-power unit.	CUDA and TensorRT, enough for 8 cameras at 3–5 fps; easy to source and repair.
OS and runtime	Ubuntu Server 24.04 LTS, Docker Engine + Compose, NVIDIA Container Toolkit	Reproducible installs and one-command upgrades.
Video	ffmpeg, PyAV, OpenCV	Already used; stream copy avoids re-encoding.
Vision	PyTorch, Ultralytics YOLO11 (TensorRT export), BoT-SORT, torchvision ResNet-50 embeddings	Matches `cow-detection` and `cow_hide_identification`.
ML	scikit-learn, LightGBM, scikit-survival, d3rlpy	Small, well-known, CPU-friendly for tabular and offline RL.
Data	PostgreSQL 16 + TimescaleDB + pgvector; Parquet for per-frame detections	One database for time series, vectors, queues and records.
Application	Python 3.11, FastAPI, Pydantic, SQLAlchemy, APScheduler	Extends the existing `app.py`.
Front end	Existing HTML/JS dashboard, extended with HTMX views and Chart.js; Arabic and English labels	No build chain; works on a farm-office PC or phone browser.
Messaging	Twilio (SMS + WhatsApp) or Meta WhatsApp Cloud API; Gammu with a USB GSM modem for offline SMS	WhatsApp business messages need pre-approved templates.
Operations	Prometheus + Grafana, systemd watchdog, restic to S3-compatible storage, WireGuard or Tailscale	Light, proven, works behind NAT.
MLOps (off the farm)	CVAT or Label Studio, DVC, MLflow, GitHub Actions for tests	Reproducible datasets and model lineage.

8 Trade-offs and decisions

Decision	We gain	We give up
Inference at the farm, not in the cloud	Works with a weak uplink, low running cost, footage stays private, intruder alerts without internet	Hardware to install and maintain on site; compute is fixed by the box
Modular monolith, not microservices	One repo, one deploy, simple debugging, fast for a small team	Modules cannot scale or be released separately; discipline is needed to keep boundaries clean
PostgreSQL as queue and store, no broker	Fewer moving parts, transactional outbox, one backup	Lower throughput ceiling than Kafka; fine for one farm but not for hundreds
Batch 60 s clips for health	Behaviour needs temporal context; reuses offline code; easier to re-run on new models	Health latency of minutes, not seconds (acceptable for this use)
Rules before learned models	Useful from week one, explainable, gives the baseline for evaluation	Lower ceiling until outcomes accumulate
Tiered identity with an unassigned state	Honest records; errors are visible and correctable	Some observations are not attributed until a person or a tag confirms them
Single node	Simple and cheap	Single point of failure; mitigated with UPS, watchdog, backups and a spare disk image

9 Risks and mitigations

Risk	Level	Mitigation
Detector misses cows in the packed feeding line and lying cows at night (already seen in the README tests)	High	Fine-tune on farm frames; IR-capable cameras; camera placement review; report metrics separately for day, night and crowded scenes.
Identity errors on solid-black cows (hide embeddings do not separate them)	High	Ear-tag OCR at chokepoints; coat-type field steers which method is trusted; unassigned state; review queue.
Too few verified health outcomes to train or evaluate the classifier and policy	High	Agree an outcome definition with the vet in week one; structured feedback in every alert; rules remain the reported baseline; describe RL results as retrospective.
Alert fatigue causes the owner to ignore alerts	Medium	De-duplication, daily digest for low priority, weekly review of false alerts, thresholds chosen with the farm.
Hardware failure or power cuts	Medium	UPS, auto-restart, health heartbeat sent by SMS, nightly backup, documented rebuild from a disk image.
GPU cannot keep up with all cameras	Medium	TensorRT FP16, adaptive frame rate (drop to 2–3 fps), process low-priority cameras at night, benchmark in M2.
Model drift with seasons, lighting or new animals	Medium	Monitor confidence and coverage per camera; monthly sample review; easy re-labelling export.
WhatsApp template approval or messaging costs	Low	Register templates early; SMS and dashboard as fallbacks.

10 Scalability and performance

8 cams	< 30 s	≤ 15 min	~30k rows/day
per GPU at 3–5 fps (target, to benchmark)	intruder alert latency	evidence to health alert	observations at 100 cows

- **Compute.** The current pipeline measured about 59 ms per frame for YOLO11m at 960 px on an Apple M3 Pro (from a run's `summary.json`). Eight cameras at 5 fps need 40 frames per second. With TensorRT FP16 on an RTX-class GPU this should fit with headroom, but the number must be confirmed in M2. The worker lowers frame rate automatically when its queue grows.
- **Storage.** One 4 MP H.265 stream at 4 Mbps is about 43 GB a day. Seven days of eight cameras is about 2.4 TB, which fits a 4 TB disk; event clips and Parquet detections are small by comparison. If the farm already has an NVR, the edge can analyse its streams and leave recording to it.
- **Herd size.** Up to roughly 300 cows per edge server; the limit is camera coverage, not the database.
- **More cameras.** Add a second GPU or a second worker node that reads from the same database.
- **More farms.** Each farm gets its own box. A later central view can pull weekly reports and anonymised features from each box over the VPN. That is the bridge to Proposal B.

11 Security and privacy

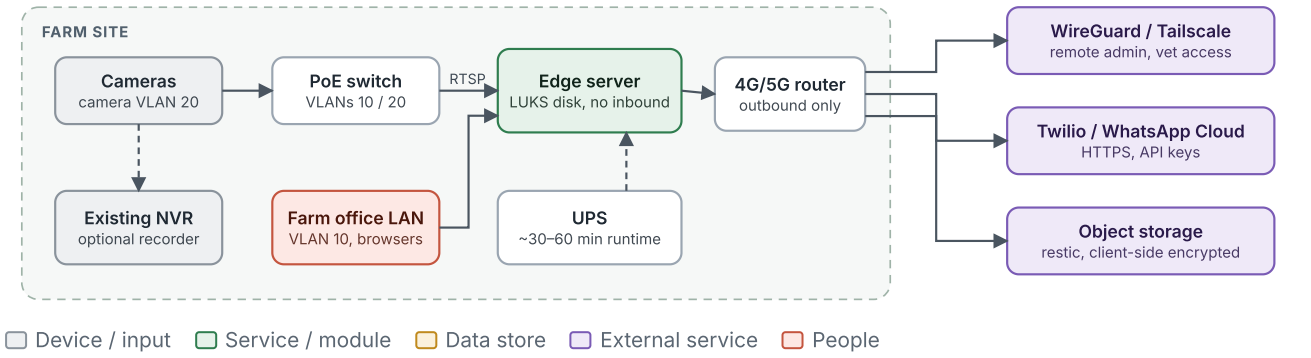


Figure 3. Network layout. Cameras sit on their own VLAN with no internet access. The edge server accepts no inbound connections from the internet.

Area	Controls
Network	Camera VLAN without internet; change default camera and NVR passwords; disable UPnP and P2P cloud features on cameras; router allows outbound only; remote access only through WireGuard or Tailscale.
Access control	Accounts with roles (owner, vet, worker, admin); vets see only their assigned farm; strong passwords plus TOTP for admin; sessions over HTTPS with a local certificate.
Secrets	RTSP credentials and API keys in Docker secrets, never in Git or config files; Twilio keys scoped to messaging.
Data at rest	Full-disk encryption (LUKS) so a stolen box does not leak footage; backups encrypted on the client by restic before upload.
Privacy	People are detected anyway by the security path, so person regions are blurred in stored and exported clips; full footage kept 7 days; signage at the farm; follow the data-protection law that applies where the farm is.

Integrity	Models are checksummed and signed before loading; every prediction stores the model and rule version; the audit log records alert acknowledgements and outcome edits.
Safety of decisions	The system only suggests; actions come from an approved list; a person approves; no automatic treatment; the RL policy runs in shadow until reviewed.

12 Cost estimate

Indicative 2026 street prices in USD for one farm. Local prices, import duties, tariffs and messaging rates vary, so treat these as planning ranges.

One-off hardware	Range
Edge server (RTX 4060 Ti 16 GB or 4070, 32 GB, 1 TB + 4 TB)	$1,300–1,900
4–8 IP cameras, 4 MP, IR (or reuse existing CCTV)	$300–900
PoE switch, cabling, mounts, enclosures	$230–450
UPS	$150–250
4G/5G router + USB GSM modem	$110–210
Total per farm	**≈ $2,100–3,600**

Monthly running cost	Range
Mobile data	$5–20
SMS + WhatsApp (under ~1,000 messages)	$5–30
Encrypted backup storage (0.5–1 TB)	$3–10
Domain / VPN (free tiers usually enough)	$0–10
Electricity (~150–250 W continuous, ≈110–180 kWh)	local tariff
Total excluding power	**≈ $20–70**

Model training uses existing laptops, university GPUs or free notebook GPU tiers; a rented cloud GPU for occasional large runs adds roughly $20–100 a month during heavy training periods. The largest real cost is team time, which this design keeps low by avoiding platform engineering.

13 Rollout plan

The phases follow the README milestones. Weeks are counted from project kick-off and overlap where packages run in parallel.

Phase	Weeks	Work	Exit criterion
0 · Contract	1–2	Choose the first target outcome and window with the vet; fix cow IDs, zones, record, prediction, alert and feedback schemas; agree alert channels and actions with the farm.	Signed-off data contract
1 · Data foundation (M1)	2–8	Install edge server and 2–4 fixed cameras (feed, water, resting); continuous recording; register cows with tags and reference photos; labelling guide; first labelled set with a second annotator; DVC versioning.	Versioned labelled dataset, split by cow and period
2 · Vision baseline (M2)	6–14	Live ingest and perception worker; fine-tuned detector; herd gallery and identity tiers; behaviour rules and posture model; security fast path; GPU benchmark.	Held-out metrics per class and condition
3 · End to end (M3)	12–20	Records and features; v0 suspicion rules; v0 priority policy; dispatcher with WhatsApp, SMS and GSM fallback; dashboard views; weekly report; one traced example from clip to outcome.	Demonstrated alert on all channels; pilot running daily
4 · Learning (M4)	18–30	Collect outcomes; v1 classifier with time window; logged decisions; offline bandit or RL policy evaluated in shadow against v0.	Retrospective evaluation report reviewed with the vet
5 · Extensions	after M4	Microphones and the audio model; cross-camera re-ID; a second farm with a lightweight central summary view; path to Proposal B if the project continues.	Decision on scale-out

14 Evaluation and success criteria

Area	Metric	Initial target (to confirm)
Detection	mAP50 on farm test frames, split into day, night and crowded feeding	Report each; no single blended number
Tracking	IDF1, HOTA, ID switches per cow-hour on annotated clips	Beat the current tracker settings
Identity	Top-1 and Recall@5 on held-out sessions; share of observations assigned	Top-1 ≥ 90% on patterned cows
Behaviour	Per-class precision and recall; minutes-per-day error versus manual annotation	Feeding and lying time within ±15%
Suspicion	Recall of vet-confirmed cases, false alerts per 100 cow-days, median lead time	Agreed with the vet before testing
Prioritisation	Off-policy value versus fixed policy; agreement with vet ranking	Non-inferior to v0 before any live use
Operations	Alert delivery time, uptime, share of clips processed in time	≥ 99% of alerts delivered within target

A Appendix · Mapping to the existing code

Existing file	Role in Proposal A
`cow-detection/detector.py`	Becomes `farmpulse.perception.detect`; adds TensorRT engine loading and the model registry.
`tracker.py`, `trackers/*.yaml`	Kept as-is for per-clip tracking; tracker state carried across consecutive clips of the same camera.
`relink.py`	Runs per clip and across clip boundaries; its calibrated thresholds feed the identity resolver.
`behaviour.py`, `zones.py`, `posture.py`	Behaviour v0 and calibration; zones move from JSON into the `zones` table, with the dashboard editor kept.
`video_processor.py`	Split into ingest (live streams) and the perception job runner; file-based CLI kept for re-processing.
`app.py`, `static/`	Grows into the API and dashboard; job endpoints remain for offline analysis.
`cow_hide_identification/src/cowid`	Training stays on the workstation; `CowEmbedder` runs in the identity resolver against the pgvector gallery.
`frame_extraction.py`	Superseded by labelling exports from the clip buffer.

Prepared from the project proposal, both README versions, the code in `cow-detection` and `cow_hide_identification`, recorded run summaries and the 13 reference papers in the project folder. Performance and cost figures are estimates to be confirmed during the milestones.